



# Tutorial 6 | OS Monsoon'25

## Section A

| Author → Rachit Bhandari



## Signals

### Motivation

Signals are a **mechanism for asynchronous communication** between processes or from the kernel to a process.

- They allow a process to **notify another process** about events (like `Ctrl+C`, timers, child termination, or custom notifications).
- Unlike shared memory, signals are **not for data transfer**, but for **control and notifications**.

### Key Concepts

- A **signal** is an integer code representing an event.
- Each signal has a **default action** (terminate, stop, ignore, etc.).
- You can **catch a signal** using a **signal handler** to perform custom actions.

### Common Signals →

| Signal               | Meaning                  |
|----------------------|--------------------------|
| <code>SIGINT</code>  | Interrupt (Ctrl+C)       |
| <code>SIGTERM</code> | Termination request      |
| <code>SIGKILL</code> | Kill (cannot be caught)  |
| <code>SIGCHLD</code> | Child process terminated |
| <code>SIGUSR1</code> | User-defined signal 1    |
| <code>SIGUSR2</code> | User-defined signal 2    |

## Code Snippet

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

// Custom signal handler
static void my_handler(int signum) {
    static int counter = 0;
    if (signum == SIGINT) {
        char buff1[23] = "\nCaught SIGINT signal\n";
        write(STDOUT_FILENO, buff1, 23);
        if (counter++ == 1) {
            char buff2[20] = "Cannot handle more\n";
            write(STDOUT_FILENO, buff2, 20);
            exit(0);
        }
    }
}
```

```

int main() {
    struct sigaction sig;
    memset(&sig, 0, sizeof(sig));
    sig.sa_handler = my_handler;

    sigaction(SIGINT, &sig, NULL);

    int n;
    while (1) {
        printf("Input a number: ");
        scanf("%d", &n);
        if (fork() == 0) { // Child process
            printf("Child: Calculating Fibonacci(%d)...\n", n);
            exit(0); // Terminate child process
        } else {
            wait(NULL); // Parent waits for child to finish
        }
    }

    return 0;
}

```

# Shared Memory

## Pre-Motivation

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#define SIZE 4

int main() {
    int A[SIZE][SIZE] = {
        {1, 0, 1, 1},

```

```

        {0, 1, 0, 1},
        {1, 1, 0, 0},
        {0, 0, 1, 1}
    };

    int B[SIZE][SIZE] = {
        {0, 1, 1, 0},
        {1, 0, 1, 1},
        {0, 1, 0, 1},
        {1, 1, 1, 0}
    };

    int C[SIZE][SIZE] = {0};
    int sum = 0; // sum of all elements in C

    printf("Parent before fork, C is all zeros\n");

    for (int r = 0; r < SIZE; r++) {
        pid_t pid = fork();
        if (pid == 0) { // child
            int local_sum = 0;
            for (int j = 0; j < SIZE; j++) {
                int val = 0;
                for (int k = 0; k < SIZE; k++)
                    val += A[r][k] * B[k][j];
                C[r][j] = val;
                local_sum += val;
            }
            printf("Child computed row %d: ", r);
            for (int i = 0; i < SIZE; i++) {
                printf("%d ", C[r][i]);
            }
            printf("\n");
            exit(local_sum & 0xFF); // send sum via exit (only works for small int
s!)
        }
    }
}

```

```

// Parent collects child results
for (int i = 0; i < SIZE; i++) {
    int status;
    wait(&status);
    if (WIFEXITED(status)) {
        sum += WEXITSTATUS(status);
    }
}

// Parent's C is unchanged due to Copy-on-Write
printf("Parent after children finish, C is still:\n");
for (int i = 0; i < SIZE; i++) {
    for (int j = 0; j < SIZE; j++) {
        printf("%d ", C[i][j]);
    }
    printf("\n");
}
printf("\n");

printf("Final sum of all C elements collected from children = %d\n", sum);
return 0;
}

```

## Motivation

When multiple processes need to **share data**, you have a few options:

- Pipes → one-way data streams
- Files → persistent storage (slow)
- **Shared Memory → fastest inter-process communication (IPC)**

Why not just use normal variables after `fork()` ?

- Each `fork()` ed child **gets a copy of the parent's memory** (Copy-On-Write).
- Any changes a child makes are **isolated** unless explicitly shared.
- Example: updating a sum variable directly in a parent will **not** reflect changes made in child processes.

Hence → we need **shared memory** for real-time, shared updates.

## Key Functions

- `shm_open(name, flags, mode)` → create/open shared memory object
- `ftruncate(fd, size)` → set the size of the memory object
- `mmap(addr, size, prot, flags, fd, offset)` → map memory into process
- `munmap(addr, size)` → unmap memory
- `shm_unlink(name)` → delete memory object

## Code Snippet

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <fcntl.h>

#define SHM_NAME "/my_shm"
#define SIZE 4

typedef struct {
    int A[SIZE][SIZE];
    int B[SIZE][SIZE];
    int C[SIZE][SIZE];
    int sum;
} shm_t;

int main() {
    // Create shared memory
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, sizeof(shm_t));
    shm_t *shm = mmap(NULL, sizeof(shm_t), PROT_READ | PROT_WRITE, M
AP_SHARED, shm_fd, 0);
```

```

// Initialize matrices
int A[SIZE][SIZE] = {
    {1, 0, 1, 1},
    {0, 1, 0, 1},
    {1, 1, 0, 0},
    {0, 0, 1, 1}
};

int B[SIZE][SIZE] = {
    {0, 1, 1, 0},
    {1, 0, 1, 1},
    {0, 1, 0, 1},
    {1, 1, 1, 0}
};

for (int i = 0; i < SIZE; i++)
    for (int j = 0; j < SIZE; j++) {
        shm→A[i][j] = A[i][j];
        shm→B[i][j] = B[i][j];
        shm→C[i][j] = 0;
    }
shm→sum = 0;

// Fork children
for (int r = 0; r < SIZE; r++) {
    if (fork() == 0) {
        int local_sum = 0;
        for (int j = 0; j < SIZE; j++) {
            int val = 0;
            for (int k = 0; k < SIZE; k++)
                val += shm→A[r][k] * shm→B[k][j];
            shm→C[r][j] = val;
            local_sum += val;
        }
        // Potential race condition here:
        // usleep(55 * abs(SIZE/2 - r));
        printf("child %d has updated sum\n", r);
        shm→sum += local_sum;
        exit(0);
    }
}

```

```

    }
}

for (int i = 0; i < SIZE; i++) wait(NULL);

printf("Final matrix C:\n");
for (int i = 0; i < SIZE; i++) {
    for (int j = 0; j < SIZE; j++) {
        printf("%d ", shm->C[i][j]);
    }
    printf("\n");
}
printf("\n");
printf("Final sum = %d (may be wrong due to race condition!)\n", shm->sum);

munmap(shm, sizeof(shm_t));
shm_unlink(SHM_NAME);
return 0;
}

```

# Semaphores

## Race Condition

A **race condition** occurs when two or more processes **access and modify shared memory concurrently**.

**Example in matrix sum:**

```
shm->sum += local_sum; // critical section
```

- If two children execute this simultaneously, the kernel might interleave the reads and writes → **lost updates**. (because what we are seeing here is not an atomic instruction)

**How to prevent** → use **semaphores**.



## Key Functions

- `sem_init(sem_t *sem, int pshared, unsigned int value)` → initialize
  - `pshared = 1` → shared across processes
  - `value` → initial count (usually 1 for mutual exclusion)
- `sem_wait(sem_t *sem)` → decrement (locks if it was initially zero)
- `sem_post(sem_t *sem)` → increment (release)
- `sem_destroy(sem_t *sem)` → cleanup

## Code Snippet

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <semaphore.h>

#define SHM_NAME "/my_shm"
#define SIZE 4

typedef struct {
    int A[SIZE][SIZE];
    int B[SIZE][SIZE];
    int C[SIZE][SIZE];
    int sum;
    sem_t lock;
} shm_t;

int main() {
    // Create shared memory
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, sizeof(shm_t));
    shm_t *shm = mmap(NULL, sizeof(shm_t), PROT_READ | PROT_WRITE, M
AP_SHARED, shm_fd, 0);
```

```

// Initialize matrices
int A[SIZE][SIZE] = {
    {1, 0, 1, 1},
    {0, 1, 0, 1},
    {1, 1, 0, 0},
    {0, 0, 1, 1}
};

int B[SIZE][SIZE] = {
    {0, 1, 1, 0},
    {1, 0, 1, 1},
    {0, 1, 0, 1},
    {1, 1, 1, 0}
};

for (int i = 0; i < SIZE; i++)
    for (int j = 0; j < SIZE; j++) {
        shm->A[i][j] = A[i][j];
        shm->B[i][j] = B[i][j];
        shm->C[i][j] = 0;
    }

shm->sum = 0;
sem_init(&shm->lock, 1, 1); // shared semaphore

// Fork children
for (int r = 0; r < SIZE; r++) {
    if (fork() == 0) {
        int local_sum = 0;
        for (int j = 0; j < SIZE; j++) {
            int val = 0;
            for (int k = 0; k < SIZE; k++)
                val += shm->A[r][k] * shm->B[k][j];
            shm->C[r][j] = val;
            local_sum += val;
        }
        sem_wait(&shm->lock);
        shm->sum += local_sum;
        sem_post(&shm->lock);
    }
}

```

```

        exit(0);
    }
}

for (int i = 0; i < SIZE; i++) wait(NULL);

printf("Final matrix C:\n");
for (int i = 0; i < SIZE; i++) {
    for (int j = 0; j < SIZE; j++) {
        printf("%d ", shm->C[i][j]);
    }
    printf("\n");
}
printf("\n");
printf("Final sum = %d (correct due to semaphore!)\n", shm->sum);

sem_destroy(&shm->lock);
munmap(shm, sizeof(shm_t));
shm_unlink(SHM_NAME);
return 0;
}

```

## [Helper] Makefile

```

all: signal basic shm shm_sem

signal:
    gcc -o signal signal.c

basic: basic.c
    gcc -o basic basic.c

shm: shm.c
    gcc -o shm shm.c

shm_sem: shm_sem.c
    gcc -o shm_sem shm_sem.c

```

run\_signal: signal

@echo "\n"

./signal

@echo "\n"

run\_shm: basic shm shm\_sem

@echo "\n"

./basic

@echo "\n"

./shm

@echo "\n"

./shm\_sem

@echo "\n"

clean:

rm -f signal

rm -f basic

rm -f shm

rm -f shm\_sem

ipc@shm: run\_shm clean

@echo "\ncompiled → run → cleaned binaries\n"

ipc@signal: run\_signal clean

@echo "\ncompiled → run → cleaned binaries\n"